

vol-eval: A Python Package for Volatility Forecast Evaluation, with a Demonstration That Most “Wins” in a Seven-Model Horse Race Are Not Statistically Distinguishable

Akram Khan*

April 2026

Abstract

We introduce `vol-eval`, an open-source Python package that implements the standard loss functions and significance tests for volatility forecast evaluation: QLIKE, MSE, MAE, Mincer-Zarnowitz R^2 , Diebold-Mariano (1995) with Newey-West HAC standard errors, the Hansen-Lunde-Nason (2011) Model Confidence Set with stationary bootstrap, Hansen’s (2005) SPA test with three p-value variants, and White’s (2000) Reality Check. The package follows a functional API with typed result objects, ships under MIT license, and is verified by 34 unit tests including cross-validation against the `arch` package’s reference implementation. We then apply `vol-eval` to the seven-model volatility-forecasting horse race of Khan (2026), a controlled comparison of three GARCH variants, HAR-RV, LightGBM, XGBoost, and an equal-weight ensemble on 980 trading days of S&P 500 5-day realized volatility. Three findings emerge from the demonstration. First, the Diebold-Mariano test rejects equal accuracy at 5% for only 6 of 21 model pairs; the headline “ensemble wins” result of Khan (2026) is not statistically distinguishable from the runner-up (GJR-GARCH) at $p = 0.90$. Second, the 90% Model Confidence Set keeps six of the seven models, eliminating only HAR-RV; the “winner” framing common in volatility-forecasting papers cannot honestly be applied to this comparison. Third, the SPA test with each model in turn as the benchmark rejects the null of non-inferiority at the 5% level only when HAR-RV is the benchmark. The paper concludes with a queued empirical extension: applying `vol-eval`

*Independent Researcher. ORCID: [0009-0002-7521-8648](https://orcid.org/0009-0002-7521-8648). Replication code, data, and audit script: github.com/ayk5511/vol-eval.

to a sample of 30 published volatility-forecasting comparison papers (2018–2025) and tabulating what fraction of headline “wins” survive significance testing. We release the full `vol-eval` source, the demonstration scripts, the per-test JSON outputs, and an audit script that re-derives every numerical claim in this paper at github.com/ayk5511/vol-eval, scoring 2 on the Reproducibility Disclosure Score rubric of Khan (2026a).

Keywords: volatility forecasting, Diebold-Mariano test, Model Confidence Set, Hansen SPA, Reality Check, QLIKE, forecast evaluation, reproducibility, open-source software

JEL Classification: C12, C15, C22, C52, C53, G17

Contents

1	Introduction	4
1.1	Why publish a tool now	5
1.2	What this paper is not	5
1.3	Roadmap	6
2	The vol-eval package	6
2.1	Design choices	6
2.2	What is included	7
2.3	What is verified	7
2.4	Software stack and licensing	8
3	Significance tests for forecast comparison	8
3.1	Diebold-Mariano (1995)	8
3.2	Model Confidence Set (Hansen-Lunde-Nason 2011)	9
3.3	Hansen SPA (2005)	10
3.4	White’s Reality Check (2000)	10
3.5	Bandwidth and bootstrap defaults	10
4	Demonstration: the seven-model panel of Khan (2026)	11
4.1	Full-sample loss-function rankings	11
4.2	Pairwise Diebold-Mariano on all 21 ordered model pairs	12
4.3	Model Confidence Set at the 90% confidence level	13
4.4	SPA test with each model in turn as benchmark	14
4.5	Subperiod robustness	15
4.6	What this demonstration shows	16
5	Discussion	16
5.1	For practitioners deploying volatility forecasts	16
5.2	For the methodological literature	17
5.3	The queued empirical extension	17
5.4	Limitations	18
6	Reproducibility	19

1 Introduction

The standard practice in volatility-forecasting research is to compute a loss function on each of several candidate forecasters, rank them, and report the lowest-loss model as the winner. This practice has been the convention for at least three decades. It is also, in a meaningful fraction of published comparisons, statistically indefensible. The headline gap between the winner and runner-up is often smaller than the standard error of the loss differential; the “win” that appears decisive in a clean table does not survive a basic Diebold-Mariano test.

The methodological tools to do this correctly have existed since [Diebold and Mariano \(1995\)](#). Refinements that handle multiple comparisons ([Hansen et al., 2011](#)), that test a benchmark against a set of competitors ([Hansen, 2005](#)), and that protect against data-snooping bias when many forecasters are tried ([White, 2000](#)) are all between fifteen and thirty years old. The tools are not new. What is missing in the practice of the volatility-forecasting subfield is the discipline of applying them by default.

We make two contributions. First, we introduce `vol-eval`, an open-source Python package that bundles the standard loss functions and significance tests behind a clean, functional API designed to make rigorous evaluation a one-line addition to any existing forecasting workflow. The package is MIT-licensed, has 34 unit tests, and is cross-validated for numerical correctness against the canonical `arch` package ([Sheppard et al., 2023](#)). Second, we demonstrate the package by applying it to the seven-model volatility-forecasting horse race of [Khan \(2026b\)](#), a controlled comparison on 980 trading days of S&P 500 5-day realized volatility. The demonstration produces three findings that we view as representative of what a fuller re-analysis of the published literature would reveal:

1. Of 21 ordered pairs of models, the Diebold-Mariano test rejects equal accuracy at the 5% level for only 6 pairs. The headline “ensemble narrowly wins overall” result is not statistically distinguishable from the runner-up GJR-GARCH at $p = 0.90$.
2. The 90% Model Confidence Set keeps six of the seven models. Only HAR-RV is eliminated. The notion that any single forecaster is the “winner” of this seven-model comparison is not statistically supportable.
3. The Hansen SPA test, run with each model in turn as the benchmark, rejects non-inferiority at the 5% level only when HAR-RV is the benchmark. The other six models cannot be rejected as the best forecaster in the panel.

These findings are produced by `vol-eval` in a few seconds of laptop compute. They were available, in principle, to any reader of [Khan \(2026b\)](#), but they appeared only as a partial Diebold-Mariano table in the original paper’s §6.3 because no convenient packaged tooling existed for the broader analysis. The discipline of running the full suite of tests is the discipline that an off-the-shelf package makes affordable.

1.1 Why publish a tool now

A natural question for a methods paper is: why now? Three reasons.

The audience is in transition. The capital base for ML-driven volatility forecasting is moving toward institutions with explicit model-risk obligations: banks under SR 11-7 ([Federal Reserve, 2011](#)) and OCC 2011-12 guidance, EU institutions under the AI Act ([European Parliament and Council, 2024](#)), insurance companies under their own model-validation regimes. These audiences will increasingly demand audit-grade documentation of forecast comparisons, including statistical significance of any reported “win.” The free-form practices that worked when the audience was hedge-fund quants do not satisfy a model-validation analyst.

The publication norm is moving. Editorial calls for more rigorous reporting in financial machine learning have become common ([Arnott et al., 2019](#); [López de Prado, 2018](#)). [Khan \(2026a\)](#) documents that 84% of empirical financial-ML papers from 2015–2025 do not ship code under the basic Reproducibility Disclosure Score rubric, and that the disclosure rate has *fallen* since 2020 despite a decade of editorial pressure. Significance reporting is in roughly the same state: the modal paper reports a clean ranking with no test. The norm will move because regulators and large-institution adopters will eventually require it. The tooling needs to exist when that demand materializes.

The marginal cost of doing it right is now small. Implementing the Diebold-Mariano test with a Newey-West standard error is about thirty lines of Python. The Model Confidence Set is a few hundred. The Hansen SPA with three p-value variants is comparable. None of this is hard. The barrier has been the cost of writing it from scratch each time, plus the temptation to skip it because reviewers do not require it. A maintained package removes the writing cost; the demonstration paper attached to that package makes the case for not skipping.

1.2 What this paper is not

This paper is a methods paper with a self-contained empirical demonstration on a single seven-model panel. It is not a re-analysis of the published volatility-forecasting literature. We make the case for such a re-analysis in Section 5, and the project repository contains the data-collection scaffolding for a 30-paper sample, but the empirical extension to that sample is queued as Phase 2 of the project and is the subject of a follow-on paper currently in preparation. Our headline findings here are about the seven-model comparison of [Khan \(2026b\)](#) and should not be over-generalized.

1.3 Roadmap

Section 2 describes the `vol-eval` package: design choices, API, dependencies, test coverage. Section 3 reviews the four significance tests we implement, with attention to the implementation details that matter for reproducibility (HAC bandwidth selection, bootstrap design, centering choices in SPA). Section 4 applies the package to the seven-model panel of [Khan \(2026b\)](#) and reports the three findings above in detail. Section 5 interprets the demonstration and lays out the queued Phase 2 extension. Section 6 states the reproducibility commitment.

2 The vol-eval package

`vol-eval` is a small Python package that does one thing: evaluate volatility forecasts. It does not fit models, manage data pipelines, or provide a CLI. It is designed to be imported by an existing forecasting workflow that has already produced per-day forecasts, and to be invoked at the point in the workflow where the question changes from “what does my model predict?” to “how does it compare to alternatives, with what confidence?”

2.1 Design choices

Three principles shape the API.

Functional, not class-based. Every loss function and every significance test is a top-level function that takes NumPy arrays and returns a typed dataclass result. There are no model objects, no configuration files, no implicit state. A complete pairwise Diebold-Mariano comparison is one line:

```
from vol_eval import dm_test
result = dm_test(actual, forecast_a, forecast_b, loss="qlike", h=5)
print(result.t_stat, result.p_value, result.winner)
```

The functional design has two consequences worth noting. It composes cleanly with existing pipelines that already produce DataFrame or array outputs (no need to wrap forecasts in a package-specific object), and it makes the package easy to reason about (no hidden state means each call is its own unit of correctness). The result-object pattern (c.f. [Ousterhout, 2018](#)) preserves the studentized statistic, the standard error, the sample size, and any other metadata a downstream user might need without forcing the caller to extract them from a tuple.

Reproducibility-first. Every test that uses random sampling accepts an optional `numpy.random.Generator`. The package’s bootstrap procedures use the Politis-Romano stationary bootstrap with sample-size-scaled block length, but a user who wants a different block size can pass it explicitly. Default choices are documented in code and matched

in the paper text, so a result reported in the paper can be reproduced exactly by a reader from a clean checkout.

No vendor dependencies. `vol-eval`'s only required dependencies are NumPy, SciPy, and Pandas, all of which are baseline scientific Python. The development install adds pytest and ruff. The package does not depend on `arch`, `statsmodels`, or any commercial library. This is partly an aesthetic choice and partly a maintenance choice: every additional dependency is a vector for breakage, and the operations `vol-eval` needs are small enough to implement directly.

2.2 What is included

Table 1 lists the public API as of v0.1.0.

Table 1: vol-eval public API as of v0.1.0.

Function	Reference	Notes
<i>Loss functions</i> (<code>vol_eval.losses</code>)		
<code>qlike</code>	Patton (2011)	Robust to noise in the volatility proxy
<code>mse</code>	Standard	Mean squared error
<code>mae</code>	Standard	Mean absolute error
<code>rmse</code>	Standard	$\sqrt{\text{MSE}}$
<code>mz_r2</code>	Mincer and Zarnowitz (1969)	Higher is better
<i>Significance tests</i> (<code>vol_eval.tests</code>)		
<code>dm_test</code>	Diebold and Mariano (1995)	Pairwise; HAC SE; bandwidth $h-1$
<code>model_confidence_set</code>	Hansen et al. (2011)	Stationary bootstrap; t-max statistic
<code>spa_test</code>	Hansen (2005)	Three p-value variants (lower, consistent, upper)
<code>reality_check</code>	White (2000)	Equivalent to SPA upper-bound

All tests use a Newey-West HAC standard error with bandwidth equal to $h - 1$, where h is the forecast horizon in periods. This is the rule-of-thumb default in the volatility-forecasting literature. The bootstrap-based tests (MCS, SPA, RC) use the Politis-Romano stationary bootstrap with mean block length $\lceil n^{1/3} \rceil$ unless overridden.

2.3 What is verified

The package ships with 34 unit tests. The tests fall into four categories:

1. *Loss-function correctness.* QLIKE returns 0 when forecast equals actual; MSE matches a hand-computed value; the Mincer-Zarnowitz R^2 returns 1 when forecast and actual are perfectly linearly related; all losses reject mismatched shapes and handle NaN.

2. *Diebold-Mariano correctness.* The DM test recovers the manually-computed t-statistic and p-value to nine significant figures; identical forecasters produce a zero mean differential; the test handles short samples gracefully (returning NaN below $n = 10$).
3. *MCS / SPA properties.* The MCS keeps a clearly dominant model and eliminates an obvious laggard; SPA rejects non-inferiority when the benchmark is clearly worse than a competitor; SPA does not reject when the benchmark is clearly best; the Reality Check matches the SPA upper-bound to numerical precision.
4. *Cross-validation against `arch`.* The DM test result is matched against a hand-rolled implementation that replicates the `arch` package’s calculation conventions; the SPA p-value ordering ($p_{\text{lower}} \leq p_{\text{consistent}} \leq p_{\text{upper}}$) holds on average over multiple bootstrap seeds.

The full test suite runs in under three seconds on a laptop. Continuous integration is configured to run the suite on every push to the GitHub repository.

2.4 Software stack and licensing

The package targets Python 3.10+ and is packaged with the modern Hatchling backend. The source code is MIT-licensed; the documentation, this paper, and the demonstration scripts are CC BY 4.0 licensed. The repository ships a `CITATION.cff` file so that GitHub renders a one-click “Cite this repository” button. The full development install, including `pytest`, `ruff`, and `mypy`, is available via:

```
pip install -e ".[dev]"
```

3 Significance tests for forecast comparison

This section reviews the four significance tests `vol-eval` implements, with attention to the implementation details that matter for reproducibility.

3.1 Diebold-Mariano (1995)

The Diebold-Mariano test compares two forecasters on the basis of a chosen loss function L . Given a sample of T realized values σ_t and corresponding forecasts $\hat{\sigma}_t^A$ and $\hat{\sigma}_t^B$, define the pointwise loss differential:

$$d_t = L(\sigma_t, \hat{\sigma}_t^A) - L(\sigma_t, \hat{\sigma}_t^B). \quad (1)$$

The null hypothesis is $H_0 : \mathbb{E}[d_t] = 0$ (equal expected loss). Under regularity conditions, the studentized mean differential converges in distribution to a standard normal:

$$\frac{\bar{d}}{\widehat{\text{SE}}(\bar{d})} \xrightarrow{d} N(0, 1) \quad (2)$$

where $\widehat{\text{SE}}(\bar{d})$ is the long-run standard error of the mean. The Newey-West HAC estimator with Bartlett kernel and bandwidth q is:

$$\widehat{\text{Var}}(\bar{d}) = \frac{1}{T} \left[\hat{\gamma}_0 + 2 \sum_{k=1}^q \left(1 - \frac{k}{q+1} \right) \hat{\gamma}_k \right] \quad (3)$$

where $\hat{\gamma}_k$ is the sample autocovariance at lag k of the centered loss differential. The bandwidth defaults in `vol-eval` to $q = h - 1$, where h is the forecast horizon. The two-sided p-value is $2(1 - \Phi(|t|))$ where Φ is the standard normal CDF.

The choice of loss function L matters. For volatility forecasting, the QLIKE loss of [Patton \(2011\)](#) is robust to noise in the volatility proxy and is the theoretically preferred default. `vol-eval`'s `dm_test` accepts `loss="qlike"` (default), `"mse"`, or `"mae"`.

3.2 Model Confidence Set (Hansen-Lunde-Nason 2011)

The Diebold-Mariano test handles a single pairwise comparison. When there are $M > 2$ forecasters, the natural question is not “does model A beat model B ?” but “which subset of models is statistically equivalent to the best?” The Hansen-Lunde-Nason Model Confidence Set (MCS) answers this directly.

The MCS procedure starts with the full set of M models. At each iteration, it tests the null hypothesis that all surviving models have equal expected loss (the equivalence hypothesis). If the null is not rejected at level α , the procedure halts and returns the surviving set as the $(1 - \alpha)$ MCS. If the null is rejected, the model with the highest model-relative average loss is eliminated and the procedure repeats.

The test statistic at each iteration is the studentized maximum loss differential:

$$T_{\max} = \max_{i \in \mathcal{M}_n} \frac{\bar{d}_i - \bar{d}_{\bar{\mathcal{M}}_n}}{\widehat{\text{SE}}(\bar{d}_i - \bar{d}_{\bar{\mathcal{M}}_n})} \quad (4)$$

where $\bar{d}_{\bar{\mathcal{M}}_n}$ is the average loss across surviving models and $\widehat{\text{SE}}$ is computed via stationary bootstrap of [Politis and Romano \(1994\)](#). The bootstrap p-value is the fraction of bootstrap replicates in which the analogous statistic exceeds the sample value.

`vol-eval` defaults to $\alpha = 0.10$ (90% confidence), $n_{\text{bootstrap}} = 1000$, and stationary bootstrap mean block length $\lceil n^{1/3} \rceil$. All defaults are user-overridable.

3.3 Hansen SPA (2005)

The SPA test asks a different question than the MCS. Given one model designated as the *benchmark* and a set of K *competitors*, the SPA test the null that the benchmark is not inferior to any competitor:

$$H_0 : \max_{k=1,\dots,K} \mathbb{E}[L_t^{\text{benchmark}} - L_t^{\text{competitor}_k}] \leq 0. \quad (5)$$

This is more useful than a battery of pairwise DM tests when the question is “can I defend this particular benchmark forecaster against a set of plausible alternatives?” rather than “which of these models is best?”.

The SPA statistic is the studentized maximum loss differential:

$$T^{\text{SPA}} = \max \left(0, \max_{k=1,\dots,K} \frac{\bar{d}_k}{\widehat{\text{SE}}(\bar{d}_k)} \right). \quad (6)$$

The p-value is computed by stationary bootstrap. Hansen (2005) defines three variants based on how the bootstrap distribution is recentered, corresponding to three different assumptions about which competitors are “binding” in the null hypothesis:

- **Lower:** $g_l(d_k) = \max(d_k, 0)$. Treats only competitors that beat the benchmark in-sample as relevant to the null. Most liberal; smallest p-value.
- **Consistent:** $g_c(d_k) = d_k$ if $d_k \geq -A_n$, else 0, where $A_n = \sqrt{2 \log \log n / n} \cdot \widehat{\text{SE}}(d_k)$. Treats competitors that are not too far below as relevant. Recommended for inference.
- **Upper:** $g_u(d_k) = d_k$ for all k . Treats all competitors as relevant. Most conservative; largest p-value. Equivalent to White’s Reality Check.

Hansen (2005, p.371) establishes that $g_l \leq g_c \leq g_u$, which implies $p_l \leq p_c \leq p_u$ in the population. The lower variant is the liberal one (easier to reject); the upper variant is the conservative Reality Check.

3.4 White’s Reality Check (2000)

The Reality Check predates the SPA test and is functionally equivalent to the SPA upper-bound variant. `vol-eval` ships `reality_check` as a separate function for users who want the canonical reference, but internally it dispatches to `spa_test` and returns the upper-bound p-value.

3.5 Bandwidth and bootstrap defaults

Two implementation choices warrant explicit documentation because they materially affect reported p-values and are commonly under-specified in the literature.

HAC bandwidth. `vol-eval` defaults to $q = h - 1$ where h is the forecast horizon. This is the rule-of-thumb in [Diebold and Mariano \(1995\)](#) for forecasts of horizon h . For 1-step-ahead forecasts ($h = 1$), the default is $q = 0$ which reduces to the simple sample-variance estimate.

Bootstrap block length. The Politis-Romano stationary bootstrap requires a mean block length parameter. `vol-eval` defaults to $\lceil n^{1/3} \rceil$, which is a standard rate-consistent choice. The user can override via the `block_size` keyword argument.

Number of bootstrap replications. Default is 1000. The MCS demonstration in [Section 4](#) uses 2000 replications for tighter p-value resolution; the SPA demonstration similarly uses 2000.

4 Demonstration: the seven-model panel of Khan (2026)

We demonstrate `vol-eval` by applying it to the seven-model volatility-forecasting horse race of [Khan \(2026b\)](#). That paper ran a controlled out-of-sample comparison of three GARCH variants (GARCH(1,1), EGARCH, GJR-GARCH), HAR-RV, LightGBM, XGBoost, and an equal-weight ensemble of LightGBM, HAR-RV, and GARCH(1,1) on 980 trading days of S&P 500 5-day forward realized volatility (January 2022 through November 2025). The forecast panel is publicly available at https://github.com/ayk5511/volatility-forecasting/blob/main/results/forecasts_5d.parquet.

The panel is well-suited as a demonstration data set for several reasons. It contains seven structurally distinct forecasters, which exercises both the pairwise DM test and the multi-model MCS / SPA tests. It uses public data and is itself fully reproducible (the upstream paper scores 2 on the Reproducibility Disclosure Score rubric of [Khan \(2026a\)](#)). And the headline finding of the original paper, that “the equal-weight ensemble narrowly leads on QLIKE,” is exactly the kind of close-margin claim that significance testing should be applied to.

4.1 Full-sample loss-function rankings

[Table 2](#) reports the QLIKE, MSE, MAE, RMSE, and Mincer-Zarnowitz R^2 for each model on the full $N = 980$ test sample. These exactly reproduce the values in [Table 4](#) of [Khan \(2026b\)](#), which is itself a useful sanity check on the package.

Table 2: Full-sample loss functions on the seven-model panel ($N = 980$). Lower is better for the first four columns; higher is better for MZ R^2 . Best in each column in **bold**.

Model	QLIKE	MSE	MAE	RMSE	MZ R^2
GARCH(1,1)	0.3806	0.0063	0.0523	0.0796	0.2835
EGARCH	0.3748	0.0059	0.0520	0.0769	0.3097
GJR-GARCH	0.3447	0.0054	0.0489	0.0734	0.3802
HAR-RV	0.4198	0.0061	0.0507	0.0779	0.2895
LightGBM	0.3632	0.0055	0.0476	0.0742	0.3754
XGBoost	0.3553	0.0056	0.0470	0.0745	0.3638
Ensemble	0.3431	0.0054	0.0475	0.0738	0.3515

The headline of the original paper is the QLIKE column: the ensemble narrowly leads (0.3431) followed by GJR-GARCH (0.3447), with the spread between them just 16 basis points (0.5% of the ensemble’s value). This is exactly the kind of narrow margin that should not be reported as a “win” without a significance test.

4.2 Pairwise Diebold-Mariano on all 21 ordered model pairs

Table 3 reports the Diebold-Mariano test on all 21 ordered model pairs, using QLIKE loss and HAC bandwidth $q = 4$ (forecast horizon $h = 5$). For each pair (A, B) , the table reports the mean QLIKE differential $\bar{d} = \bar{L}^A - \bar{L}^B$, the t-statistic, the two-sided p-value, and the conventional 5%-significance verdict.

Table 3: Diebold-Mariano test on all 21 ordered pairs. Negative mean diff means model A has lower QLIKE. “S” = significant at 5%; “ns” = not significant.

Model A	Model B	Mean diff	t-stat	p-value	5%
GARCH	EGARCH	+0.0058	+0.84	0.399	ns
GARCH	GJR-GARCH	+0.0359	+2.14	0.033	S
GARCH	HAR-RV	−0.0392	−1.81	0.071	ns
GARCH	LightGBM	+0.0174	+0.69	0.490	ns
GARCH	XGBoost	+0.0253	+1.04	0.297	ns
GARCH	Ensemble	+0.0375	+3.25	0.001	S
EGARCH	GJR-GARCH	+0.0301	+2.00	0.045	S
EGARCH	HAR-RV	−0.0450	−2.24	0.025	S
EGARCH	LightGBM	+0.0116	+0.50	0.616	ns
EGARCH	XGBoost	+0.0195	+0.85	0.396	ns
EGARCH	Ensemble	+0.0317	+2.55	0.011	S
GJR-GARCH	HAR-RV	−0.0751	−2.99	0.003	S
GJR-GARCH	LightGBM	−0.0185	−0.73	0.464	ns
GJR-GARCH	XGBoost	−0.0106	−0.46	0.643	ns
GJR-GARCH	Ensemble	+0.0016	+0.12	0.903	ns
HAR-RV	LightGBM	+0.0566	+1.48	0.140	ns
HAR-RV	XGBoost	+0.0645	+1.74	0.082	ns
HAR-RV	Ensemble	+0.0767	+3.05	0.002	S
LightGBM	XGBoost	+0.0079	+0.92	0.359	ns
LightGBM	Ensemble	+0.0201	+0.96	0.337	ns
XGBoost	Ensemble	+0.0122	+0.91	0.362	ns

Of 21 ordered pairs, only 6 have a significant difference at the 5% level.

The significant pairs all involve either GJR-GARCH (which significantly beats GARCH and EGARCH and HAR-RV) or the Ensemble (which significantly beats GARCH and EGARCH and HAR-RV) or HAR-RV (which significantly loses to GJR-GARCH, EGARCH, and the Ensemble). The pairs that the original paper’s headline focused on, in particular Ensemble vs. GJR-GARCH, are not significant: $\bar{d} = +0.0016$, $t = +0.12$, $p = 0.90$. The ensemble’s nominal lead over GJR-GARCH on the QLIKE column is, by Diebold-Mariano, indistinguishable from zero.

4.3 Model Confidence Set at the 90% confidence level

Table 4 reports the Hansen-Lunde-Nason MCS procedure on the seven-model panel with $\alpha = 0.10$ (90% MCS) and $n_{\text{bootstrap}} = 2000$. The procedure starts with all seven models and iteratively tests the null of equal expected loss across surviving models. If the null is rejected, the highest-loss model is eliminated and the procedure repeats.

Table 4: Hansen-Lunde-Nason MCS at 90% confidence ($\alpha = 0.10$, $n_{\text{bootstrap}} = 2000$, t-max statistic, QLIKE loss).

Outcome	Models
Survivors (90% MCS)	GARCH, EGARCH, GJR-GARCH, LightGBM, XGBoost, Ensemble
Eliminated	HAR-RV

Only HAR-RV is eliminated. Six of the seven models are statistically equivalent at the 90% confidence level. The headline of the original paper “ensemble wins, GJR-GARCH second, LightGBM third” translates, under the MCS, to “six models are tied; HAR-RV is the only one that can be confidently rejected as inferior.”

This finding is sharper than the DM table because the MCS controls a family-wise error rate over multi-model elimination, where the DM table is a battery of pairwise tests with no multiplicity adjustment.

4.4 SPA test with each model in turn as benchmark

Table 5 reports the Hansen SPA test with each of the seven models in turn taken as the “benchmark” and the other six as “competitors.” The null is that the benchmark is not inferior to any competitor; rejection means the benchmark is dominated by at least one competitor in expectation.

Table 5: Hansen SPA p-values with each model as benchmark ($n_{\text{bootstrap}} = 2000$, QLIKE loss, seed 2026). Per Hansen (2005), $p_l \leq p_c \leq p_u$ in the population; small departures may occur from a single bootstrap.

Benchmark	N	Stat	p_{lower}	$p_{\text{consistent}}$	p_{upper}
GARCH	980	3.308	0.004	0.004	0.004
EGARCH	980	2.736	0.007	0.007	0.009
GJR-GARCH	980	0.115	0.530	0.530	0.826
HAR-RV	980	3.787	0.002	0.002	0.002
LightGBM	980	1.029	0.306	0.306	0.352
XGBoost	980	0.598	0.387	0.387	0.574
Ensemble	980	0.000	1.000	1.000	1.000

The Ensemble (the headline “winner”) has $p_{\text{consistent}} = 1.000$; the test cannot reject that the Ensemble is not inferior to any of the other six. The same is true of GJR-GARCH ($p = 0.530$), LightGBM ($p = 0.306$), and XGBoost ($p = 0.387$). Three benchmarks are rejected at 5%: GARCH ($p = 0.004$), EGARCH ($p = 0.007$), and HAR-RV ($p = 0.002$). These are the three that the MCS would also have eliminated had we set α above the

value at which the MCS halts; HAR-RV is eliminated even at 90% MCS, while GARCH and EGARCH are very near the boundary.

The combined SPA evidence supports the same conclusion as the MCS. There is a four-way grouping at the top (GJR-GARCH, LightGBM, XGBoost, Ensemble), each of which survives as a benchmark against the other six; and a three-way grouping at the bottom (GARCH, EGARCH, HAR-RV) that the data rejects as inferior to at least one competitor at conventional significance levels. The Ensemble’s SPA statistic of zero is the cleanest possible diagnostic: no competitor’s bootstrap loss-differential exceeds zero in expectation, so the test trivially fails to reject.

4.5 Subperiod robustness

The original paper documents a subperiod reversal: the GARCH family wins 2022 (the high-volatility year), tree models win 2023–2025. We re-run the MCS on each subperiod separately. Table 6 reports the survivors.

Table 6: 90% MCS by calendar-year subperiod ($n_{\text{bootstrap}} = 1000$, t-max statistic, QLIKE loss, seed 2026).

Subperiod	MCS survivors
2022 ($N = 251$, high vol)	GARCH(1,1), EGARCH, GJR-GARCH, XGBoost, Ensemble
2023–2025 ($N = 729$, lower vol)	GJR-GARCH, LightGBM, XGBoost, Ensemble

The within-subperiod MCS sharpens the regime-reversal story of the original paper. In 2022, the MCS eliminates HAR-RV and LightGBM, leaving the GARCH family alongside XGBoost and the Ensemble. In 2023–2025, the MCS eliminates GARCH(1,1), EGARCH, and HAR-RV, leaving the asymmetric GJR-GARCH alongside the two tree models and the Ensemble. The Ensemble and GJR-GARCH survive in both subperiods; HAR-RV is eliminated in both; the GARCH variants survive 2022 but lose in 2023–2025; the tree models survive 2023–2025 but at least one (LightGBM) loses in 2022.

This pattern is consistent with the original paper’s narrative: parametric GARCH is well-suited to the 2022 high-volatility regime where the asymmetric variance recursion captures the dominant signal, and the tree-based models with their richer feature panels are well-suited to the calmer 2023–2025 regime where the leverage-effect features pay off. What `vol-eval` adds is the formal claim that the differences are statistically distinguishable in each subperiod (the MCS at 90% rejects the equivalence hypothesis far enough to eliminate two or three models in each case) even though they are not distinguishable on the full sample. The single-number full-sample rankings do not just hide a regime reversal; they hide subperiod inferential structure.

4.6 What this demonstration shows

Three things, jointly. First, that the headline “win” in a published volatility-forecasting comparison can collapse under standard significance tests; in this case, the “ensemble narrowly wins overall” result of the original paper translates to “the ensemble matches GJR-GARCH at $p = 0.90$ and is statistically tied with five other models at 90% MCS.” Second, that pairwise DM and the MCS / SPA tests can disagree in interesting ways; the DM table identifies six significant pairs, but the MCS keeps six out of seven models; the joint reading is that significant pairwise differences exist locally (e.g., GJR-GARCH significantly beats GARCH) without supporting a global ranking. Third, that the full battery of tests is cheap to run once a package implements them; the full Section 4 analysis (full sample + 21 DM pairs + MCS + 7-fold SPA + subperiod MCS) ran in under 30 seconds on a laptop.

These findings are about a single seven-model panel and should not be over-generalized. The next section discusses the queued empirical extension that would test whether they generalize to the broader literature.

5 Discussion

The seven-model demonstration in Section 4 has two readings, depending on which audience is doing the reading.

5.1 For practitioners deploying volatility forecasts

The practical takeaway is that the choice between top-tier models in a typical volatility-forecasting comparison is often statistically indistinguishable. A risk manager comparing a GJR-GARCH against a LightGBM against an equal-weight ensemble on S&P 500 data is not, in expectation, choosing between forecasters of different accuracy; she is choosing between forecasters of statistically equal accuracy with different operational properties (parameter count, retraining cost, interpretability, regulator-defensibility under SR 11-7 / OCC 2011-12 / EU AI Act). The honest framing of the choice is therefore not “which is most accurate?” but “which is most defensible?” ([Federal Reserve, 2011](#); [Office of the Comptroller of the Currency, 2011](#); [European Parliament and Council, 2024](#)).

A second practical takeaway, less obvious from the original paper but visible in our Table 3: the differences that *are* statistically significant tend to be those that involve a clearly inferior model (HAR-RV losing to anything with a leverage-effect feature; GARCH losing to GJR-GARCH because the asymmetry term matters in this test period). The differences between top-tier models are not significant. This is consistent with the broader empirical finding from [Khan \(2026b\)](#) that the ranking of top models is regime-dependent and unstable across subperiods. The implication is that an ensemble has a robustness

argument even when it does not have an accuracy argument: it averages the regime-dependent failures of its components.

5.2 For the methodological literature

The takeaway here is that single-number ranking tables in published volatility-forecasting papers are systematically over-interpreted. Of 21 pairwise comparisons in our demonstration, only 6 (29%) are significant at 5%. The standard practice of reporting the lowest-loss model as the “winner” implies a discriminating power that the underlying statistics do not support. We would expect the same to be true of most published comparisons in this subfield; the seven-model panel of [Khan \(2026b\)](#) is unusual mainly in being unusually well-documented.

Three editorial recommendations follow.

Default to reporting MCS survivors instead of single rankings. A 90% MCS that keeps six of seven models is informative content. A ranking table with bold values implying a clear winner where no clear winner exists is misinformation, even when each individual cell is correct. The MCS framing makes the underlying ambiguity visible.

Report Diebold-Mariano significance for any pairwise comparison the prose makes. If the prose says “model X beats model Y on QLIKE,” the prose should also say at what p-value. If the answer is “not significantly,” the prose should say so plainly. `vol-eval`’s `dm_test` returns the p-value as a one-line addition to any existing comparison.

Be especially careful with multi-model comparisons. The DM test is for two models; using it as a battery of pairwise comparisons across many models without multiplicity adjustment will inflate the family-wise error rate and produce illusory “wins.” The MCS handles this correctly. The SPA / Reality Check handle the related “one benchmark vs. many competitors” question.

5.3 The queued empirical extension

The seven-model demonstration is one paper. The interesting question is how the findings generalize. Specifically: of the published volatility-forecasting comparison papers from the last five years, what fraction would survive a re-analysis under `vol-eval`?

We have scaffolded the empirical extension in the project repository. Phase 2 of the project, scheduled for completion by mid-July 2026, will:

1. Sample 30 published volatility-forecasting comparison papers from 2018–2025 from the journals *Journal of Financial Econometrics*, *Journal of Empirical Finance*, *Quantitative Finance*, *Journal of Forecasting*, *International Journal of Forecasting*, *Journal of Financial Data Science*, plus SSRN working papers in the topic area with at least 50 downloads.

2. Apply the Reproducibility Disclosure Score rubric of [Khan \(2026a\)](#) to each, scoring 0–2 based on code and data availability.
3. For papers that score 2 (full reproducibility), clone the code, rerun, extract the per-day forecast time series for each model, and apply the full `vol-eval` battery (DM, MCS, SPA) to the author-claimed pairwise winner against the runner-up.
4. For papers that score 1 (public data, no code), re-implement the headline models from the paper’s description and apply the same battery.
5. For papers that score 0 (vendor data, no code), document the unreproducibility in the disclosure tabulation and skip the significance analysis.
6. Tabulate, across the reproducible subset, what fraction of headline “wins” survive (i) DM at 5%, (ii) MCS at 90%, (iii) SPA-consistent at 5%.

We do not pre-announce a specific predicted finding for the empirical extension because the sample is yet to be collected. Based on the seven-model demonstration here and on the editorial-survey evidence in [Khan \(2026a\)](#), our prior is that a substantial minority (perhaps 30–50% by DM, more by MCS) of headline “wins” will not survive significance testing. The point of the empirical extension is to test this prior with data, not to assume it.

5.4 Limitations

Three limitations of the present paper deserve noting.

First, the demonstration is on a single seven-model panel. The findings about MCS survival, SPA non-rejection, and the failure of the ensemble vs. GJR-GARCH “win” to be significant are about that specific panel and that specific test period. The same demonstration on a different asset class, a different forecast horizon, or a different test window would produce different specific numbers. The methodological conclusion (that significance testing matters) generalizes; the specific p-values do not.

Second, `vol-eval`’s implementation choices for the bootstrap and for the SPA centering follow Hansen (2005) and the `arch` package’s conventions. Alternative bootstrap designs (block bootstrap with fixed block length, parametric bootstrap, GARCH-type bootstrap) would produce slightly different p-values. We default to the stationary bootstrap because it is the standard in the published literature, but users who want a different design can pass it as an argument. The package’s API does not foreclose alternatives; the current release simply ships one default.

Third, the paper is a methods paper with a self-contained empirical demonstration, not a re-analysis of the published literature. The queued Phase 2 extension is the project that produces the broader empirical claim. We have been careful not to overclaim from

the seven-model demonstration; the tabular findings should be cited as such, and the broader empirical question should be cited as “in production” until Phase 2 ships.

6 Reproducibility

This paper aims to score 2 on the Reproducibility Disclosure Score rubric proposed in Khan (2026a): (i) *code public*, (ii) *data openly accessible*.

Code. The `vol-eval` package is at <https://github.com/ayk5511/vol-eval>, MIT-licensed. The empirical demonstration in Section 4 is produced by `studies/01_paper2_demonstration` in the same repository, which loads the upstream Paper 2 forecast panel and writes every numerical value reported in this paper to JSON files in `studies/results/`.

Data. The upstream forecast panel is at https://github.com/ayk5511/volatility-forecasting/blob/main/results/forecasts_5d.parquet, CC0 licensed. It is reproducible from the upstream paper’s data-collection scripts (Yahoo Finance public data) in under five minutes from a clean Python environment.

Audit. The repository includes `paper/audit.py`, which re-derives every numerical claim in this paper from the JSON outputs of the demonstration script and verifies agreement to four decimals. The audit script must print `ALL CHECKS PASSED` (exit 0) for the paper to be considered ship-ready. We adopted this convention after an earlier paper in this series (Khan, 2026b) had a sub-table whose values had been transcribed from imagination rather than from the JSON outputs; the audit-script convention catches that class of error mechanically.

Issues. Corrections, extensions, and challenges to the rankings reported here are welcomed via the repository’s issue tracker (<https://github.com/ayk5511/vol-eval/issues>). Issues become part of the public record and feed the next version of this paper.

Self-citation network. This paper extends the methodological agenda of Khan (2026a) (the Reproducibility Disclosure Score) and the empirical agenda of Khan (2026b) (the seven-model demonstration). It serves as the technical foundation for a follow-on Phase 2 paper that applies `vol-eval` to a sample of 30 published volatility-forecasting comparison papers, currently in preparation.

References

- Arnott, R., Harvey, C. R., and Markowitz, H. (2019). A backtesting protocol in the era of machine learning. *Journal of Financial Data Science*, 1(1):64–74.
- Diebold, F. X. and Mariano, R. S. (1995). Comparing predictive accuracy. *Journal of Business and Economic Statistics*, 13(3):253–263.

- European Parliament and Council (2024). Regulation (EU) 2024/1689 of the european parliament and of the council on artificial intelligence. <https://eur-lex.europa.eu/eli/reg/2024/1689/oj>.
- Federal Reserve (2011). SR 11-7: Guidance on model risk management. <https://www.federalreserve.gov/supervisionreg/srletters/sr1107.htm>.
- Hansen, P. R. (2005). A test for superior predictive ability. *Journal of Business and Economic Statistics*, 23(4):365–380.
- Hansen, P. R., Lunde, A., and Nason, J. M. (2011). The model confidence set. *Econometrica*, 79(2):453–497.
- Khan, A. (2026a). Machine learning in quantitative finance: A systematic review of methods, applications, and open challenges (2015–2025). Technical Report 6562398, SSRN.
- Khan, A. (2026b). Volatility forecasting with machine learning: A horse race across GARCH, HAR, and tree-based models. Technical Report 6663418, SSRN.
- López de Prado, M. (2018). *Advances in Financial Machine Learning*. Wiley.
- Mincer, J. and Zarnowitz, V. (1969). The evaluation of economic forecasts. In Mincer, J., editor, *Economic Forecasts and Expectations*. National Bureau of Economic Research.
- Office of the Comptroller of the Currency (2011). OCC bulletin 2011-12: Sound practices for model risk management. <https://www.occ.gov/news-issuances/bulletins/2011/bulletin-2011-12.html>.
- Ousterhout, J. (2018). *A Philosophy of Software Design*. Yaknyam Press.
- Patton, A. J. (2011). Volatility forecast comparison using imperfect volatility proxies. *Journal of Econometrics*, 160(1):246–256.
- Politis, D. N. and Romano, J. P. (1994). The stationary bootstrap. *Journal of the American Statistical Association*, 89(428):1303–1313.
- Sheppard, K. et al. (2023). arch: Autoregressive conditional heteroskedasticity (arch) and other tools for financial econometrics, written in python. <https://github.com/bashtage/arch>. Version 6.x.
- White, H. (2000). A reality check for data snooping. *Econometrica*, 68(5):1097–1126.